

Destructuring

Destructuring is a way to unpack values from arrays or objects the separate variables in a concise and readable manner.

Array Destructuring

Syntax

```
const [a, b] = [1, 2];
```

Examples

```
const fruits = ["Apple", "Oranges",  
"Bananas", "Mangos"];
```

```
let [fruit1, fruit2] = fruits
```

```
console.log(fruit1, fruit2);
```

Output:

Apple Oranges

Skipping Array Values

```
const [fruit1, , , fruit4] = fruits;  
console.log(fruit1, fruit4);
```

Output:

Apple Mangos

Array Position Values

```
let { [0]: fruit1, [2]: fruit3 } = fruits;
```

Output:

Apple Bananas

The Rest Property

This syntax will store all remaining values into a new array

```
const num = [1, 2, 3, 4, 5, 6, 7, 8];
```

```
const [a, b, ...rest] = num;
```

Output:

a: 1 b: 2 rest: 3, 4, 5, 6, 7, 8

Swapping JS variables

```
let firstName = "John";  
let lastName = "Doe";
```

```
[firstName, lastName] = [lastName,  
                           firstName];
```

Use Case

Quickly extract values from arrays especially from API responses.

Object Destructuring:

Syntax

```
const {key1, key2} = objectName;
```

Examples

```
const person = {
```



```
firstName = "John",  
lastName = "Doe",  
age = "25",
```

```
};
```

```
let { firstName, lastName } = person;
```

2]; The order of property does not matter

```
let { lastName, firstName } = person;
```

Object Default Values

```
let { firstName, lastName, country = "US" }  
= person;
```

Nested Destructuring

```
const student = {
```

```
  name: "John",
```

```
  marks: { math: 90, science: 85 }
```

```
};
```



```
const {marks: {math, science}} =  
  student;
```

```
console.log(math, science);
```

Output:

90 85

```
const arr = [1, [2, 3]];
```

```
const [a, [b, c]] = arr;
```

```
console.log(b, c);
```

Output:

2 3

Destructuring in Function Parameter

```
function greet({name, age}) {  
  console.log("Hello", name);  
}
```



```
function sum([a, b]) {  
    return a + b;  
}  
console.log(sum([2, 3]));
```

Template Literals

Template literals provide a more powerful way to work with strings using backticks (`). They support interpolation, multi-line strings and embedded expressions.

Backtick

```
const msg = 'This is a string';
```

String Interpolation

```
'Hello ${variable}'
```



```
const name = Shumaila;
```

```
console.log(`Hello, ${name}!`);
```

```
// Hello, Shumaila
```

```
const a = 5, b = 10;
```

```
console.log(`Sum: ${a+b}`);
```

```
// Sum: 15
```

Multi-line Strings

```
const poem = `
```

```
Roses are red,
```

```
Violets are blue,
```

```
Grass is green.`;
```

```
console.log(poem);
```


Expression Evaluation

```
const price = 100;
```

```
const discount = 20;
```

```
console.log('Final Price:  
$ { price - discount }');
```